EE789
Aa Assignment
Hemant Hajare 20D070037

Question 1

prog.aa

next_out

It acts as an accumulator and accumulates the sum after every multiplication. Since the final product of 8-bit numbers can be 16-bit, next_out is made 16 bit long.

a_ in, b_in

These are temporary variables in the loop, loopback. They are initialized to a, b respectively on entry and assigned next_ a, next_b next iteration onwards a_in, b_in will be updated in the process.

$volatile continue_flag := (b_in >0)

This is the stopping condition for loop, “loopback”. In every iteration b_in is a right shifted and assigned to next_b. At the end of multiplication b_in in will assume a value of 0 and thus it is the stopping condition.

$volatile b_bit := (b_in [] 0)

This captures the bit value at the LSB after every right shift. It can be 0 or 1 based on which $volatile add will be determined. Add stores a_in or 0 if b_bit is 0 or 1 respectively. This is captured in $volatile add := ($mux b_bit a_in 0) where a_in 0 are appropriately casted to 16-bit. We append next_out with add as next_out := (next_out + add).

a_in wil left shifted by 1 every time b_in is right shifted and thus next_b := (next_b >> 1)
and next_a := (next_a << 1).

Once b_in assumes 0 we step out of the loop and assign the product value of next_out.

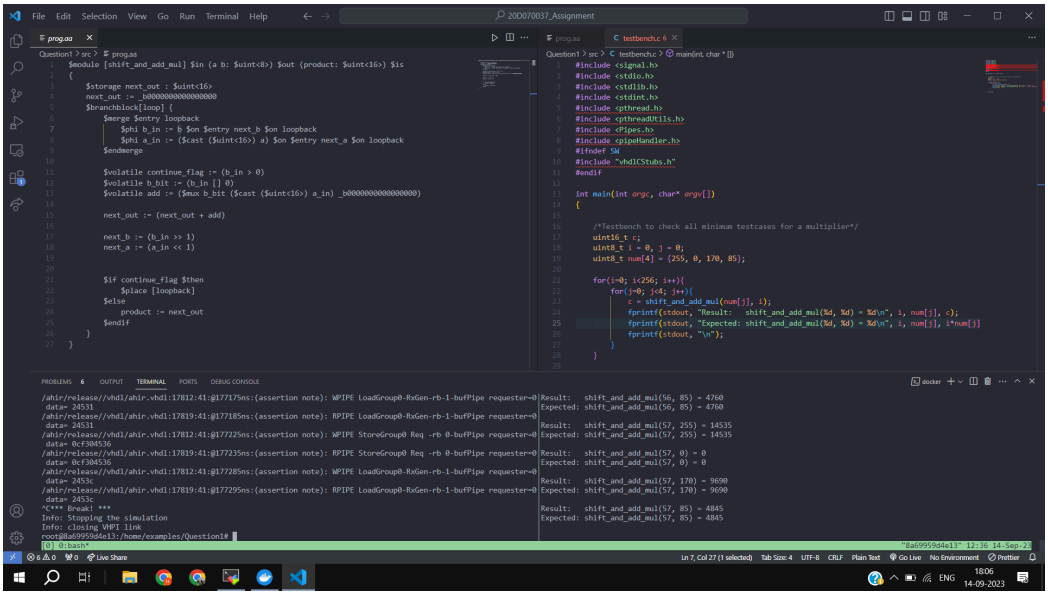
testbench.c

The number of testcases to verify the circuit is 256*256 = 65536. However, even though feasible for 8_bit numbers we test only a few minimal test cases by some rudimentary arguments.

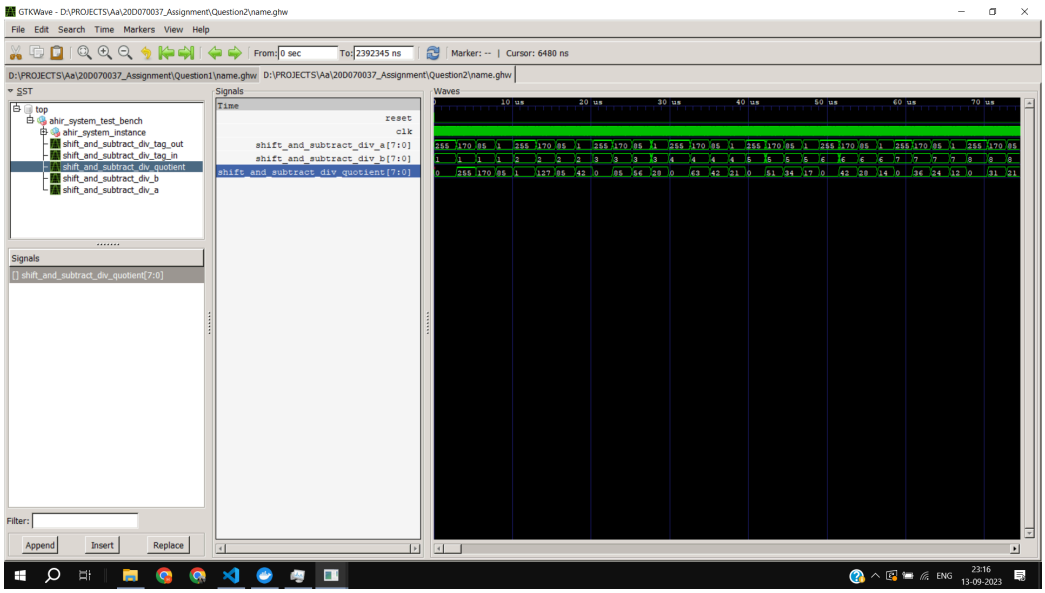
We multiply each of the numbers from 0-255 with 4 numbers whose binary representations are

1. 1 1 1 1 1 1 1 1 = 255
2. 0 0 0 0 0 0 0 0 = 0
3. 1 0 1 0 1 0 1 0 = 170
4. 0 1 0 1 0 1 0 0 = 85

Testcase 1. checks if a can be multiplied correctly with 1 at all places. testcase 2. checks if a can be multiplied correctly by 0 at all the places. testcase 3. check if the shifting operation works correctly with 1 at even places and testcase 4. checks if the shifting operation works correctly with 1 at odd places.



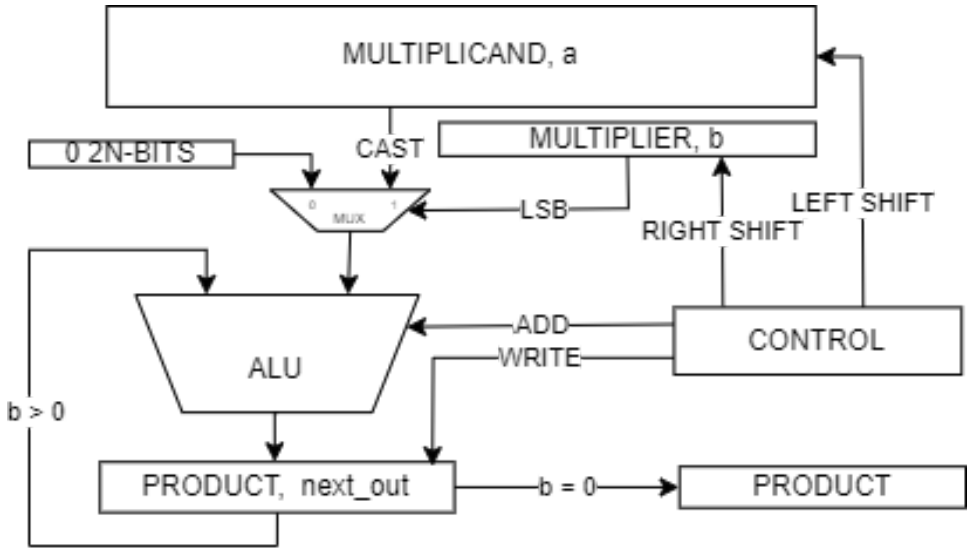
GHDL Simulation for shift and add multiplier



GTKWAVE for shift and add multiplier

Following modules were identified in the generated VHDL file.

1. Block for Phi Operator
2. Block for MUX
3. Block for Casting
4. Load-Store Blocks
5. Branch Blocks
6. Input and Output Buffer



Datapath for shift and multiplier as per the VHDL code